Curso
Tendencias Aplicadas de Ingenería de Software

Persistencia de Datos No Relacional

Persistencia de Datos No Relacional

- La persistencia de datos constituye uno de los pilares fundamentales de la ingeniería de software moderna. Durante mucho tiempo, las bases de datos relacionales han sido la tecnología predominante para almacenar información debido a su sólido fundamento teórico, basado en el álgebra relacional y en las propiedades ACID.

- Sin embargo, la aparición de aplicaciones web masivas, plataformas globales, sistemas distribuidos, redes sociales, servicios de streaming, soluciones cloud e Internet de las Cosas (IoT) ha generado nuevos requerimientos que desafían las capacidades tradicionales de los sistemas relacionales.

Persistencia de Datos No Relacional

- En este contexto surgen las bases de datos NoSQL (Not Only SQL), un conjunto de tecnologías de persistencia que priorizan aspectos como la escalabilidad horizontal, la disponibilidad, la flexibilidad de los esquemas y el manejo eficiente de grandes volúmenes de información distribuidos geográficamente.

- Es importante destacar que NoSQL no pretende reemplazar completamente al modelo relacional. Más bien, propone alternativas que resultan más adecuadas para determinados escenarios arquitectónicos.

- La selección de una tecnología de persistencia debe realizarse considerando los requisitos funcionales y no funcionales del sistema, así como las restricciones operativas y de negocio.

Persistencia de Datos No Relacional

- Durante las décadas de 1980, 1990 y comienzos de los años 2000, la mayoría de las aplicaciones empresariales operaban sobre bases de datos relacionales. Este modelo funcionaba adecuadamente porque los volúmenes de información eran relativamente moderados y las aplicaciones se ejecutaban normalmente dentro de una única organización o centro de datos.

- La situación cambió radicalmente con la aparición de aplicaciones que procesan grandes volumenes de datos y de usuarios.

- Estas organizaciones descubrieron que los enfoques tradicionales de escalamiento vertical resultaban insuficientes o extremadamente costosos.

Persistencia de Datos No Relacional

- Además, comenzaron a surgir nuevos tipos de datos:
  - Contenido multimedia.
  - Registros de eventos (logs).
  - Datos de sensores IoT.
  - Información geoespacial.
  - Datos semiestructurados en formatos JSON y XML.
  - Información generada por usuarios en redes sociales.
- Las nuevas aplicaciones requerían almacenar y procesar volúmenes masivos de información en tiempo real, manteniendo altos niveles de disponibilidad.
- Como respuesta a estos desafíos, aparecieron nuevas tecnologías de persistencia agrupadas bajo el término NoSQL.

Limitaciones del Modelo Relacional

- Aunque los sistemas relacionales continúan siendo extremadamente importantes, presentan ciertas limitaciones cuando se utilizan en entornos altamente distribuidos o de gran escala.
- Uno de los principales problemas es la dependencia de esquemas rígidos.
- En una base de datos relacional, cualquier modificación estructural requiere modificar tablas, índices, restricciones y, en ocasiones, migrar millones de registros existentes.
- Este proceso puede afectar la disponibilidad y el rendimiento del sistema.

Limitaciones del Modelo Relacional

- Otro desafío importante es la necesidad de realizar JOINs para reconstruir relaciones complejas.
- Cuando las tablas contienen millones o miles de millones de registros, estas operaciones pueden transformarse en cuellos de botella significativos.
- Desde el punto de vista de la escalabilidad, los sistemas relacionales suelen crecer mediante escalamiento vertical, es decir, incorporando más memoria, procesadores o almacenamiento al mismo servidor.
- Sin embargo, esta estrategia presenta límites físicos y económicos.
- Finalmente, los mecanismos de consistencia transaccional estricta, aunque muy valiosos para aplicaciones financieras o críticas, pueden generar dificultades cuando se intenta distribuir datos entre múltiples centros de datos geográficamente separados.

Sistemas Distribuidos y Nuevos Desafíos

- Uno de los desafíos de los sistemas modernos no es almacenar datos, sino hacerlo de manera confiable en ambientes distribuidos.
- En un sistema distribuido pueden ocurrir múltiples tipos de fallas:
  - Caída de servidores.
  - Interrupciones de red.
  - Latencias variables.
  - Pérdida temporal de conectividad.
  - Inconsistencias entre réplicas.
  - Sobrecarga de algunos nodos.

Sistemas Distribuidos y Nuevos Desafíos

- A diferencia de un sistema centralizado, donde todos los componentes comparten memoria y almacenamiento, en una arquitectura distribuida cada nodo posee una visión parcial del sistema.

- Esto obliga a resolver problemas complejos relacionados con:
  - Replicación de datos.
  - Sincronización.
  - Consistencia.
  - Recuperación ante fallas.
  - Balanceo de carga.

- Muchas tecnologías NoSQL fueron diseñadas específicamente para enfrentar estos desafíos desde su origen, incorporando mecanismos de particionamiento y replicación distribuidos de forma nativa.

Motivaciones Fundamentales de NoSQL

- Las soluciones NoSQL no surgieron simplemente como una alternativa tecnológica, sino como una respuesta a necesidades concretas de las organizaciones modernas.
- Entre los principales objetivos se encuentran:

Escalabilidad horizontal

- Permitir el crecimiento mediante la incorporación de nuevos servidores en lugar de depender exclusivamente de hardware cada vez más potente y costoso.

Alta disponibilidad

- Mantener el servicio operativo incluso cuando algunos nodos o enlaces de comunicación fallen.

Motivaciones Fundamentales de NoSQL

Flexibilidad estructural

- Facilitar la evolución de los modelos de datos sin requerir complejas migraciones de esquema.

Rendimiento

- Optimizar operaciones de lectura y escritura para aplicaciones que procesan millones de transacciones por segundo.

Distribución geográfica

- Permitir que los datos se encuentren replicados en distintas regiones para reducir latencia y mejorar la resiliencia.
- Estas características resultan especialmente relevantes para arquitecturas cloud, sistemas de microservicios y aplicaciones globales.

Flexibilidad de Datos y Evolución de Esquemas

- Uno de los aspectos más atractivos de muchas bases de datos NoSQL es su capacidad para manejar estructuras heterogéneas dentro de una misma colección de datos.
- Por ejemplo, en una base documental como MongoDB es posible almacenar documentos con diferentes atributos sin necesidad de redefinir el esquema global.
- Esto resulta particularmente útil cuando:
  - Los requisitos cambian frecuentemente.
  - El dominio de negocio evoluciona rápidamente.
  - Existen múltiples versiones de objetos.
  - Se integran fuentes de datos diversas.

Flexibilidad de Datos y Evolución de Esquemas

- Desde la perspectiva de la ingeniería de software, esta flexibilidad reduce el costo de evolución del sistema y facilita la adopción de enfoques ágiles.
- No obstante, también introduce desafíos. La ausencia de restricciones estructurales estrictas puede provocar inconsistencias, duplicación de datos o dificultades de mantenimiento si no se aplican adecuadas prácticas de modelado.
- Por este motivo, la flexibilidad no debe interpretarse como ausencia de diseño. Por el contrario, exige una comprensión profunda de los patrones de acceso y de los requisitos del dominio.

Ventajas de NoSQL Frente al Modelo Relacional

- Las bases de datos NoSQL ofrecen ventajas significativas en determinados escenarios arquitectónicos.
- En primer lugar, proporcionan mecanismos de escalabilidad horizontal que permiten distribuir la información entre múltiples nodos. Esto posibilita manejar volúmenes de datos que resultarían difíciles de administrar mediante un único servidor.
- En segundo lugar, muchas tecnologías NoSQL están optimizadas para operar en entornos distribuidos y cloud-native, incorporando capacidades de replicación y recuperación automática ante fallas.

Ventajas de NoSQL Frente al Modelo Relacional

- Otra ventaja importante es la reducción de la complejidad asociada a las operaciones JOIN. En muchos modelos NoSQL, los datos se almacenan de forma desnormalizada, priorizando la eficiencia de acceso.
- Asimismo, permiten trabajar con datos semiestructurados y no estructurados de manera más natural que las bases relacionales tradicionales.
- Sin embargo, estas ventajas suelen obtenerse a costa de sacrificar algunos aspectos del modelo relacional, como la consistencia fuerte, las transacciones complejas o la integridad referencial estricta.
- Por esta razón, la elección entre SQL y NoSQL debe basarse en los requisitos específicos del sistema y no en tendencias tecnológicas.

Teorema CAP (Consistency, Availabity, Partition tolerance)

- Uno nuevo concepto que puede llegar a incidir en el diseño de bases de datos distribuidas es el Teorema CAP.
- El teorema establece que un sistema distribuido no puede garantizar simultáneamente tres propiedades cuando ocurre una partición de red:
- Consistency (C)
  - Todos los nodos observan exactamente la misma información al mismo tiempo. Una lectura realizada inmediatamente después de una escritura siempre devuelve el valor más reciente.
- Availability (A)
  - Toda solicitud recibe una respuesta válida, aunque algunos nodos del sistema hayan fallado.

Teorema CAP (Consistency, Availabity, Partition tolerance)

- Partition Tolerance (P)
  - El sistema continúa funcionando incluso cuando existen fallas de comunicación entre grupos de nodos.
- En ambientes distribuidos reales, las particiones de red son inevitables. Por esta razón, los arquitectos deben decidir qué característica priorizar cuando dichas fallas ocurren.
- El teorema CAP no implica que una base de datos deba elegir permanentemente sólo dos propiedades, sino que durante una partición será necesario realizar compromisos entre consistencia y disponibilidad.

Teorema CAP (Consistency, Availabity, Partition tolerance)

- El verdadero valor del teorema CAP no reside en la teoría matemática sino en sus consecuencias prácticas para el diseño arquitectónico.

- Supongamos un sistema distribuido compuesto por tres centros de datos ubicados en diferentes regiones del mundo. Si una interrupción de red impide la comunicación entre ellos, aparecen dos opciones:

- **Mantener la consistencia**
  - El sistema rechaza algunas solicitudes hasta que pueda garantizar que todos los nodos poseen la misma información.

- Consecuencias:
  - Menor disponibilidad.
  - Mayor confiabilidad de los datos.
  - Posibles tiempos de espera.
  - Mantener la disponibilidad

Teorema CAP (Consistency, Availabity, Partition tolerance)

- **Mantener la disponibilidad**
  - Cada nodo continúa respondiendo solicitudes aunque no pueda sincronizarse inmediatamente con los demás.

- Consecuencias:
  - Mejor experiencia de usuario.
  - Riesgo de lecturas desactualizadas.
  - Necesidad de mecanismos posteriores de reconciliación.

- Las bases de datos relacionales tradicionales suelen privilegiar la consistencia, mientras que muchos sistemas NoSQL fueron diseñados para favorecer la disponibilidad y la tolerancia a particiones.

Teorema CAP (Consistency, Availabity, Partition tolerance)

- Un ejemplo sencillo: Supongamos que se tienen dos servidores:
  - Servidor A en Montevideo.
  - Servidor B en Madrid.
- Ambos almacenan la misma información.
- Ahora un usuario cambia su nombre en el servidor A, pero justo en ese momento se corta la comunicación entre A y B (una partición de red).
- Según CAP, cuando ocurre esa partición, debes elegir entre:
- **Opción 1: Consistencia (CP):** El servidor B deja de responder o rechaza ciertas consultas hasta sincronizarse con A.
- En esta opción:
  - Los datos son siempre correctos y consistentes.
  - Algunas solicitudes pueden no recibir respuesta.

Teorema CAP (Consistency, Availabity, Partition tolerance)

- **Opción 2: Disponibilidad (AP):** El servidor B sigue respondiendo aunque tenga datos antiguos.
- En esta opción:
  - El sistema siempre responde.
  - Durante un tiempo algunos usuarios pueden ver información diferente.
- **¿Por qué no existe CA?**
- Existen 3 combinaciones posibles
  - CA
  - CP
  - AP
- Pero en sistemas distribuidos reales la P (tolerancia a particiones) es prácticamente obligatoria, porque los fallos de red pueden ocurrir.
- Por eso, cuando hay una partición, normalmente la decisión real es: ¿Se prefiere consistencia (CP) o disponibilidad (AP)?

Teorema CAP (Consistency, Availabity, Partition tolerance)

- Una analogía rápida: Supongamos dos sucursales de una biblioteca.
  - Consistencia: ambas tienen exactamente el mismo catálogo.
  - Disponibilidad: ambas atienden a los usuarios siempre.
- Partición: se cae Internet entre las sucursales.
- Si una sucursal presta un libro y no puede avisar a la otra:
  - Si se prioriza consistencia, una sucursal deja de prestar libros hasta sincronizarse.
  - Si se prioriza disponibilidad, ambas siguen prestando libros, aunque durante un tiempo el catálogo pueda diferir.
- No puedes tener las tres cosas perfectas al mismo tiempo cuando existe una partición.
- Resumen: El teorema CAP dice que, en un sistema distribuido, cuando ocurre un fallo de comunicación entre nodos, debes elegir entre mantener la consistencia de los datos o mantener la disponibilidad

Consistencia Eventual

- La consistencia eventual es uno de los conceptos más importantes dentro del ecosistema NoSQL.
- La idea central consiste en aceptar que diferentes nodos puedan contener versiones distintas de un mismo dato durante un período limitado de tiempo.
- Por ejemplo:
  - Un usuario modifica su perfil.
  - El cambio se registra en un nodo.
  - Las réplicas reciben la actualización gradualmente.
  - Durante algunos segundos diferentes usuarios podrían observar versiones distintas.
- Finalmente todas las réplicas convergen al mismo estado.
- Este comportamiento permite:
  - Reducir latencias.
  - Mejorar disponibilidad.
  - Incrementar capacidad de escalamiento.

Consistencia Eventual

- Sin embargo, también introduce desafíos.
- Los desarrolladores deben considerar escenarios donde:
  - Una lectura puede devolver información antigua.
  - Existen escrituras concurrentes.
  - Aparecen conflictos entre réplicas.
- Muchos sistemas modernos implementan variantes más sofisticadas como:
  - Read-after-write consistency.
  - Session consistency.
  - Strong eventual consistency.
- Comprender estos modelos resulta esencial para diseñar aplicaciones distribuidas confiables.